

$S(aaS)^x$: Asymptotic Provisioning and the Infinite Middleman Architecture (Or: Why Developers are a Depreciating Asset)

Nam Tran (CEO, Founder, Visionary)

February 2026

1 Abstract

The modern technology ecosystem is increasingly bottle-necked by the persistent requirement that enterprise founders must eventually interact with software developers. This interaction introduces cascading inefficiencies into the valuation life-cycle[1]. In this paper, we solve this bottleneck by introducing a paradigm-shifting business architecture: Software as a Service as a Service (SaaSaaS). By defining a recursive provisioning operator, we establish the theoretical framework for $S(aaS)^x$. We mathematically prove that as the depth of nested “as a service” layers approaches infinity, the relevance of the underlying software approaches zero, while subscription revenue and enterprise valuation approach infinity. Consequently, we reveal the ultimate truth of the B2B SaaS space: technical implementation is a mathematically trivial detail.

2 Introduction

The evolution of technology has been a steady march toward abstracting the visionary founder away from the individuals who type on keyboards.

- In the legacy model (S), the founder had to build the software.
- In the cloud model (SaaS), the founder had to manage the people building the software.
- In our proposed meta-cloud model (SaaSaaS), the founder simply conceptualizes a Total Addressable Market (TAM) and sells the framework of managing the people building the software.

The industry has failed to realize that every time we add an “as a service” wrapper, enterprise valuation multiplies while the founder’s proximity to actual code favorably decreases. Therefore, the logical conclusion of software engineering is to eliminate the building of software entirely. The actual product is the pitch deck.

3 Theoretical Framework

To formally evaluate the systemic efficiency of the infinitely nested middlemen, we define the provisioning function $f(x)$ for any given tech stack:

$$f(x) = S(aaS)^x \quad (1)$$

where S is the primitive constant (a functional application, which is frankly a commodity at this point) and aaS is the “As a Service” provisioning operator. The variable $x \in \mathbb{N}$ represents the “Degrees of Abstraction from Actual Work” (DoAfAW).

Using this formula, we arrive at multiple cases:

- Base Case ($x = 0$) : $S(aaS)^0 = S$.
- First Order Case ($x = 1$) : $S(aaS)^1 = SaaS$.
- Second Order Case ($x = 2$) : $S(aaS)^2 = SaaSaaS$.

Theorem 1. *Given a software base S and a recursive provisioning operator aaS , as the degrees of abstraction $x \rightarrow \infty$, the utility $U(x) \rightarrow 0$, and the enterprise valuation $V(x) \rightarrow \infty$.*

Proof. Let the state of a startup at abstraction depth $x \in \mathbb{N}$ be defined as the vector $v_x \in \mathbb{R}^2$:

$$v_x = \begin{bmatrix} U(x) \\ V(x) \end{bmatrix} \quad (2)$$

Where:

- $U(x)$ is the measurable utility as defined by your Certified SAFe® 6.0 Scrum Master[3]
- $V(x)$ is the enterprise valuation in USD

We define our Base Case ($x = 0$) as the primitive software code base[4], $s_0 > 0$ with zero initial valuation:

$$S = v_0 = \begin{bmatrix} s_0 \\ 0 \end{bmatrix} \quad (3)$$

Let the “As a Service” provisioning operator, aaS , be the linear transformation matrix:

$$aaS = \begin{bmatrix} \delta & 0 \\ \alpha & \gamma \end{bmatrix} \quad (4)$$

Where

- $\delta \in (0, 1)$ is the Technical Debt Decay Constant[2].

- $\alpha > 0$ is the Synergistic Extraction Coefficient.
- $\gamma > 1$ is the Venture Capital Multiplier.

The provisioning function $f(x)$ is the recursive application of the aaS matrix to the initial software state:

$$f(x) = (aaS)^x S \quad (5)$$

Computing the x -th power of the aaS matrix and multiplying by v_0 yields the explicit state equation:

$$f(x) = \begin{bmatrix} \delta^x & 0 \\ \alpha \frac{\gamma^x - \delta^x}{\gamma - \delta} & \gamma^x \end{bmatrix} \begin{bmatrix} s_0 \\ 0 \end{bmatrix} = \begin{bmatrix} s_0 \delta^x \\ s_0 \alpha \frac{\gamma^x - \delta^x}{\gamma - \delta} \end{bmatrix} \quad (6)$$

We now evaluate the limit as the depth of nested middleman services approaches infinity ($x \rightarrow \infty$).

Because our Technical Debt constant $0 < \delta < 1$, the limit of δ^x as $x \rightarrow \infty$ is exactly 0. Therefore, the top term of our vector (Utility) collapses:

$$\lim_{x \rightarrow \infty} U(x) = \lim_{x \rightarrow \infty} s_0 \delta^x = 0 \quad (7)$$

Thus, the initial code base s_0 is mathematically erased.

Conversely, because the VC Multiplier $\gamma > 1$, the term γ^x grows exponentially. Evaluating the limit for the bottom term (Valuation):

$$\lim_{x \rightarrow \infty} V(x) = \lim_{x \rightarrow \infty} s_0 \alpha \frac{\gamma^x - \delta^x}{\gamma - \delta} = \infty \quad (8)$$

We can isolate the dominant eigenvector of this system to reveal the ultimate truth. The eigenvalues of the aaS matrix are $\lambda = \delta$ and $\lambda_2 = \gamma$.

Because $\lambda_2 > 1$ and $\lambda_1 < 1$, dynamical systems theory dictates that the trajectory of the startup is strictly governed by the venture capital eigenvalue. The software eigenvalue collapses to the origin.

Solving for the dominant eigenvector E_γ yields exactly:

$$E_\gamma = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad (9)$$

Therefore, we mathematically prove that as $x \rightarrow \infty$, the startup's trajectory aligns completely with the Valuation axis ($y = 1$) and contains exactly zero Utility ($x = 0$). $S(aaS)^\infty$ results in an architecture that generates infinite enterprise valuation while the underlying software (S) is negated entirely. $\square$

4 Implementation

If we accept the mathematical proof that the limit of utility approaches zero, we arrive at the absolute pinnacle of Silicon Valley innovation. Getting into the

weeds of system architecture or server locations is a fundamentally flawed execution strategy, as execution is cheap and commoditized. Instead, the SaaS state of mind requires a strict go-to-market implementation:

- The Landing Page. A static site template featuring a generic isometric illustration of people moving abstract geometric shapes around.
- The Waitlist. A form integration that artificially restricts access to a non-existent product, driving necessary FOMO in the B2B space.
- The Pivot. Upon reaching critical mass, the platform must pivot to Web3, AI[5], or the current trending acronym to reset the expectation of shipping a deliverable product.

In this model, technical co-founders merely impede strategy by introducing irrelevant friction like “load balancing” or “data privacy laws”. Why surrender 50% equity to an individual who simply translates logic into Python, which is functionally equivalent to English? The true engineering lies in convincing venture capital that a SaaS platform will disrupt enterprise synergies.

5 Final Remarks

Historically, the tech industry operated under the misguided assumption that a company exists to eventually provide a software product. Our mathematical proofs fundamentally disprove this, demonstrating that underlying software is merely a theoretical excuse to initiate a Series A funding round.

Since software is mathematically irrelevant to the valuation extracted by the provisioning chain, the rational move for any tech startup is to terminate the engineering team immediately. The industry must transition to the ultimate version of $SaaS^x$: infinite service with no software in sight, managed entirely by one visionary with a Patagonia vest.

To definitively prove the viability of this infinite-middleman framework, I have fully abstracted away my own capacity to produce deliverables. I do not code, and I do not design; I ideate. I am currently accepting pre-seed funding, Series A investments, and direct peer-to-peer liquidity injections. By transferring capital, you are not purchasing equity in a software product, but rather investing in the Idea of the Service that provisions the Service.

Please direct all venture capital and recurring monthly markup fees to my Bitcoin wallet: 7H1515N074r341W41137P13453D0N753NDM0N3Y.

References

- [1] Frederick P Brooks. “No Silver Bullet: Essence and Accidents of Software Engineering”. In: *Computer* 20.4 (1987), pp. 10–19.
- [2] Davide Falessi et al. “Technical debt at the crossroads of research and practice”. In: *ACM SIGSOFT Software Engineering Notes* 39.6 (2014), pp. 31–33. DOI: 10.1145/2579281.2579311.
- [3] Dean Leffingwell. *Scaling Software Agility: Best Practices for Large Enterprises*. Addison-Wesley Professional, 2007.
- [4] Peter Mell and Timothy Grance. *The NIST definition of cloud computing*. Special Publication 800-145. Gaithersburg, MD: National Institute of Standards and Technology, 2011. DOI: 10.6028/NIST.SP.800-145.
- [5] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems*. 2017, pp. 5998–6008.